INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Clasificación de dígitos manuscritos con MNIST
Unidad de Organización Curricular:	Profesional
Nivel y Paralelo:	Octavo / A
Alumnos participantes:	Fiallos Yanza Jonathan Josue Guevara Panimboza Steven Marlon
Asignatura:	Inteligencia Artificial
Docente:	Rubén Nogales

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

- Comparar el desempeño de un árbol de decisión, Random Forest y k-NN al clasificar dígitos manuscritos de MNIST mediante un protocolo común de entrenamiento y prueba.

Específicos:

- Preparar las imágenes de MNIST y reproducir los conjuntos de entrenamiento y prueba mediante muestreo estratificado representativo.
- Ajustar y evaluar los tres clasificadores con exactamente los mismos datos y calcular su exactitud y error en cada conjunto.
- Justificar la elección de un modelo a partir de las métricas de prueba, el análisis de la brecha de generalización y el estudio de sus matrices de confusión.

2.2 Modalidad

Práctica en parejas, con ejecución en laboratorio y elaboración del informe fuera de clase.

2.3 Tiempo de duración

- **Presenciales:** 2 horas para preparar los datos, entrenar y evaluar.
- **No presenciales:** 1 hora para comparar resultados y elaborar el informe.

2.4 Instrucciones

Completen los datos de portada y registren los nombres en orden alfabético. Usen Python y conserven un solo cuaderno de trabajo por pareja. Mantengan la misma partición, transformación e hiperparámetros indicados para que los resultados puedan compararse.

Registren el entorno y las versiones de Python y scikit-learn. Completen las tablas con los resultados de su ejecución y numeren las figuras y tablas del informe.

2.5 Listado de equipos, materiales y recursos

- Computador con conexión a Internet para la descarga inicial de MNIST [1].
- Jupyter Notebook o Google Colab con Python, NumPy, pandas, scikit-learn y Matplotlib.
- **TAC:** Cuaderno interactivo para ejecutar código, visualizar imágenes y documentar los resultados.

2.6 Actividades por desarrollar

Actividad 1. Preparar MNIST. Se utilizó la base de datos de referencia `mnist_784`, versión 1 (OpenML, ID 554) [1], construida por Yann LeCun, Corinna Cortes y Christopher Burges a partir de muestras de dígitos manuscritos de la oficina del censo estadounidense (NIST SD-3) y estudiantes de secundaria (NIST SD-1). Cada imagen consta de una resolución matricial de 28×28 píxeles en escala de grises, totalizando 784 variables por observación, con valores originales entre 0 (fondo negro) y 255 (trazo blanco), y una etiqueta entera entre 0 y 9. En la Figura 1 se presentan cinco muestras del dataset junto con sus etiquetas verdaderas.

Figura 1: Muestras representativas del conjunto MNIST (28x28 píxeles)

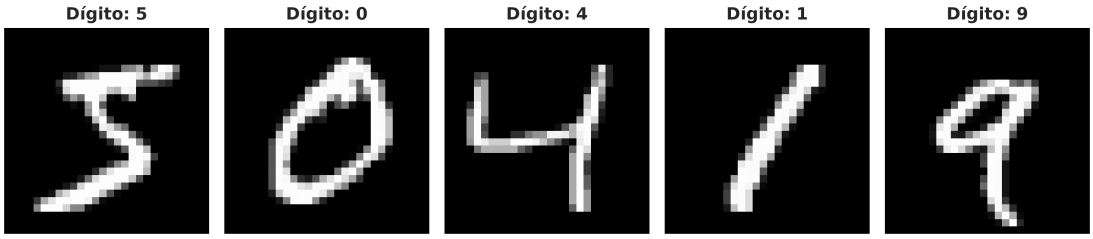


Figura 1: Visualización de cinco muestras representativas del conjunto MNIST y sus etiquetas.

Para el desarrollo experimental se conservó rigurosamente la partición canónica del reservorio:

- Las primeras 60 000 observaciones corresponden al conjunto de entrenamiento original.
- Las últimas 10 000 observaciones corresponden al conjunto de prueba original.

A partir de dichos bloques, se extrajeron subconjuntos representativos de $N_{\text{ent}} = 10\,000$ instancias para entrenamiento y $N_{\text{pru}} = 2\,000$ instancias para prueba aplicando un muestreo aleatorio estratificado según el dígito con semilla fija `random_state=42`. Esto garantiza que la probabilidad a priori de cada clase $P(Y = k)$ permanezca balanceada e idéntica en ambos subconjuntos.

Cada imagen se transformó en un vector unidimensional $x \in [0, 1]^{784}$ mediante normalización lineal:

$$x_j = \frac{p_j}{255.0}, \quad \forall j \in \{1, 2, \ldots, 784\}$$

donde p_j denota la intensidad entera del píxel j. Las dimensiones matriciales resultantes son:

$$X_{\text{ent}} \in \mathbb{R}^{10\,000 \times 784}, \quad y_{\text{ent}} \in \mathbb{Z}^{10\,000}$$

$$X_{\text{pru}} \in \mathbb{R}^{2\,000 \times 784}, \quad y_{\text{pru}} \in \mathbb{Z}^{2\,000}$$

En la Tabla 1 y en la Figura 2 se ilustra la distribución de frecuencias absolutas y relativas por clase para ambos conjuntos.

Cuadro 1: Distribución de muestras por dígito en entrenamiento y prueba.

Dígito	Ejemplos Ent.	Prop. Ent. (%)	Ejemplos Pru.	Prop. Pru. (%)
0	987	9.87	196	9.80
1	1124	11.24	227	11.35
2	993	9.93	206	10.30
3	1022	10.22	202	10.10
4	974	9.74	196	9.80
5	904	9.04	178	8.90
6	986	9.86	192	9.60
7	1044	10.44	206	10.30
8	975	9.75	195	9.75
9	991	9.91	202	10.10
Total	**10 000**	**100.00 %**	**2 000**	**100.00 %**

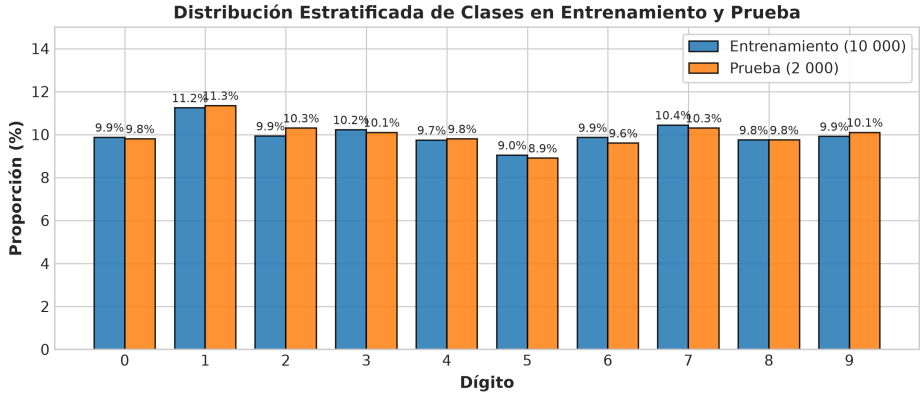


Figura 2: Distribución estratificada de dígitos en los conjuntos de entrenamiento (10 000) y prueba (2 000).

Actividad 2. Entrenar. Se instanció un objeto para cada modelo bajo la configuración parametrizada en la Tabla 2. Cada clasificador fue ajustado mediante el método `fit(X_ent, y_ent)` sobre exactamente los mismos datos de entrenamiento.

Cuadro 2: Configuración e hiperparámetros fijados para los clasificadores.

Modelo	Clase de scikit-learn	Hiperparámetros fijados
Árbol de decisión	`DecisionTreeClassifier` [2]	`max_depth=15, random_state=42`
Random Forest	`RandomForestClassifier` [3]	`n_estimators=100, max_depth=15, random_state=42`
k-NN	`KNeighborsClassifier` [4]	`n_neighbors=5, metric="euclidean", weights="uniform"`

Fundamentación Teórica del Método `fit` en Cada Modelo:

- **Árbol de Decisión (CART):** Durante la invocación de `fit`, el algoritmo induce de forma voraz y recursiva una jerarquía binaria de partición del espacio muestral $\mathbb{R}^{784}$. En cada nodo interno,

selecciona el par (j, θ) compuesto por una característica de píxel j y un umbral de corte $\theta \in [0, 1]$ que maximice la reducción de impureza de Gini:

$$\Delta I_G = I_G(D) - \left(\frac{|D_L|}{|D|} I_G(D_L) + \frac{|D_R|}{|D|} I_G(D_R) \right)$$

donde el índice de Gini se define como $I_G(D) = 1 - \sum_{k=0}^{9} p_k^2$, siendo p_k la proporción de muestras de la clase k en el nodo. El proceso se detiene forzosamente cuando un nodo alcanza una profundidad de 15 niveles (`max_depth=15`).

- **Random Forest (Ensamble por Bagging):** Al invocar `fit`, construye de forma paralela un bosque compuesto por 100 árboles de decisión individuales (`n_estimators=100`). Cada árbol $t \in \{1, \ldots, 100\}$ se entrena con una muestra generada mediante remuestreo con reemplazo (*bootstrap aggregating*) de tamaño $N_{\text{ent}} = 10\,000$. En cada bifurcación, a diferencia de un árbol tradicional, el algoritmo no explora las 784 variables completas, sino que restringe la búsqueda a un subespacio aleatorio de tamaño $m = \lfloor \sqrt{784} \rfloor = 28$ características. Esto descorrelaciona la estructura de los árboles individuales y abate la varianza del ensamble:

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^{B} T_b(x) \right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

reduciendo sustancialmente el término $\frac{1-\rho}{B}\sigma^2$ a medida que $B = 100$ crece y disminuyendo la correlación inter-árbol ρ.

- **k-NN (k-Nearest Neighbors):** Al pertenecer al paradigma de aprendizaje basado en instancias o aprendizaje perezoso (*lazy learning*), el método `fit` no calcula coeficientes analíticos ni genera una función discriminante paramétrica explícita. Su función consiste únicamente en indexar y organizar los 10 000 vectores de entrenamiento en estructuras jerárquicas espaciales (como KD-Trees o Ball-Trees) en la memoria principal para habilitar búsquedas de vecinos próximos eficientes durante la fase de predicción.

Actividad 3. Evaluar. Para cada clasificador se aplicó el método `predict` de forma independiente sobre los conjuntos X_{ent} y X_{pru}, contrastando las predicciones contra las etiquetas reales para computar:

$$\text{Exactitud (\%)} = 100 \times \frac{\sum_{i=1}^{N} \mathbb{I}(y_i = \hat{y}_i)}{N} = \texttt{accuracy_score}(y, \hat{y}) \times 100$$

$$\text{Error de clasificación (\%)} = 100 - \text{Exactitud (\%)}$$

$$\text{Brecha (p. p.)} = \text{Exactitud}_{\text{ent}} - \text{Exactitud}_{\text{pru}}$$

Actividad 4. Comparar. Se construyeron matrices de confusión sobre el conjunto de prueba para las 10 clases ($C \in \mathbb{Z}^{10\times10}$):

$$C_{ij} = \sum_{k=1}^{N_{\text{pru}}} \mathbb{I}(y_k = i \wedge \hat{y}_k = j)$$

donde la fila i representa la clase verdadera y la columna j representa la predicción emitida. A partir de ello se identificaron con exactitud las peores confusiones interclase ($i \neq j$) y se analizaron las causas morfológicas de los fallos.

2.7 Resultados obtenidos

Registro del entorno computacional utilizado:

- **Intérprete de Python:** Versión 3.12.4 (64-bit, MSC v.1940 AMD64).
- **Librería scikit-learn:** Versión 1.9.1.
- **Librerías de soporte:** NumPy 2.x, pandas 2.x, Matplotlib 3.x, SciPy.

En la Tabla 3 se presentan los resultados cuantitativos consolidados de exactitud, error y brecha de generalización para los tres modelos.

Cuadro 3: Resultados de evaluación en entrenamiento y prueba.

Modelo	Exactitud ent. (%)	Error ent. (%)	Exactitud prueba (%)	Error prueba (%)	Brecha (p. p.)
Árbol de decisión	99.02	0.98	81.80	18.20	17.22
Random Forest	99.93	0.07	94.80	5.20	5.13
k-NN	96.51	3.49	94.55	5.45	1.96

Nota: Ent. = entrenamiento; p. p. = puntos porcentuales. Se verificó que en cada conjunto la exactitud y el error sumen exactamente el 100.00 %.

En la Figura 3 se aprecia visualmente la relación entre el rendimiento en entrenamiento y prueba junto con la brecha de sobreajuste. Asimismo, en la Figura 4 se ilustra el análisis de tiempos computacionales de ajuste (`fit`) e inferencia (`predict`).

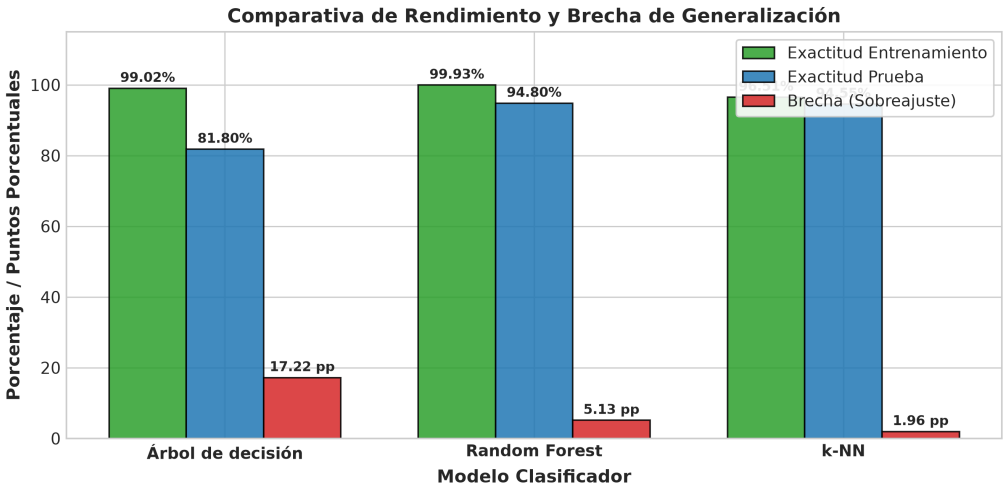


Figura 3: Comparativa cuantitativa de exactitud en entrenamiento vs prueba y brecha de sobreajuste.

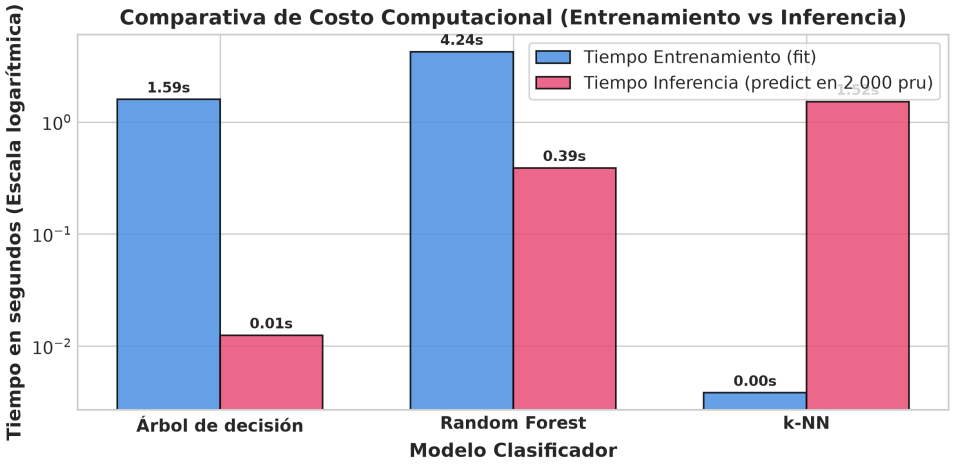


Figura 4: Costo computacional de entrenamiento e inferencia en escala logarítmica para los tres clasificadores.

Análisis de las Matrices de Confusión. En la Figura 5 se presentan las matrices de confusión de prueba para los tres clasificadores evaluados. Adicionalmente, en la Figura 6 se desglosan las matrices individuales con barras de escala cromática y valores enteros normalizados.

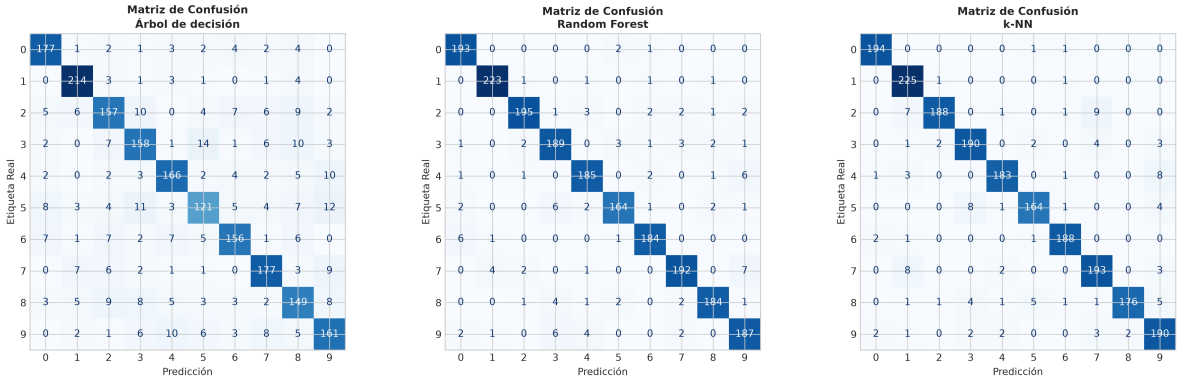
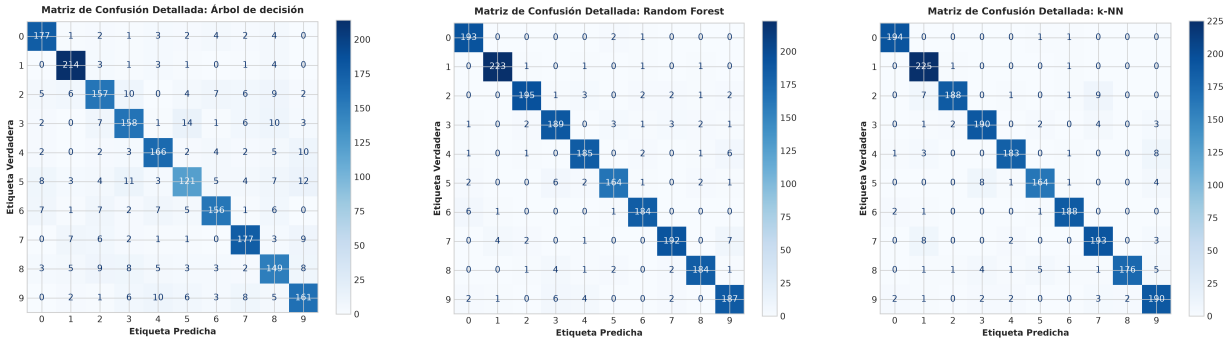


Figura 5: Matrices de confusión sobre el conjunto de prueba (2 000 ejemplos) para Árbol de Decisión, Random Forest y k-NN.



(a) Árbol de Decisión (b) Random Forest (c) k-NN ($k = 5$)

Figura 6: Matrices de confusión individuales detalladas con valores cuantitativos por celda.

Al evaluar las celdas fuera de la diagonal principal ($i \neq j$), se determinaron las confusiones más

frecuentes entre dígitos distintos para cada modelo:

- **Árbol de decisión:** La mayor confusión ocurrió entre el **dígito real 3 clasificado erróneamente como 5**, registrando un total de **14 casos** (seguido por 4 vs 9 con 11 casos y 7 vs 9 con 11 casos). Esto obedece a que el árbol simple divide el espacio con cortes ortogonales rígidos paralelos a los ejes de coordenadas de píxel individuales, siendo incapaz de modelar la continuidad curvilínea de los arcos inferiores compartidos por el 3 y el 5.
- **Random Forest:** La confusión más frecuente se presentó con el **dígito real 7 clasificado erróneamente como 9**, sumando un total de **7 casos**. Dicha ambigüedad surge cuando el trazo superior horizontal de un 7 manuscrito posee una inclinación o densidad que cierra parcialmente la parte superior semejando el lazo cerrado característico del 9.
- **k-NN:** La mayor confusión se observó en el **dígito real 2 clasificado erróneamente como 7**, alcanzando **9 casos**. Al fundamentarse estrictamente en la distancia euclidiana en un espacio de 784 dimensiones (afectado por la maldición de la dimensionalidad), trazos de 2 con base horizontal tenue o curva presentan un vector de distancias acumuladas muy próximo a los patrones canónicos del 7.

Análisis de Muestras de Prueba Mal Clasificadas. En la Figura 7 se expone la muestra de prueba correspondiente al índice 64, cuya etiqueta real es el dígito 2. En dicha instancia se aprecian las siguientes predicciones emitidas:

- **Etiqueta Real:** 2
- **Predicción Árbol de Decisión:** 6
- **Predicción Random Forest:** 4
- **Predicción k-NN:** 2 (acierto correcto)

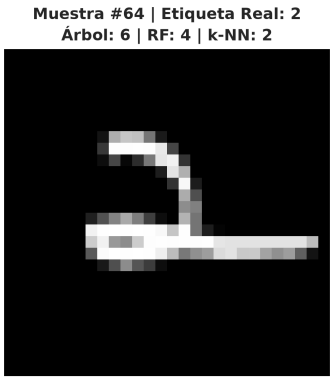


Figura 7: Muestra de prueba #64 con caligrafía atípica: etiqueta real 2, predicciones: Árbol=6, Random Forest=4, k-NN=2.

Como se observa, el trazo caligráfico posee un lazo superior comprimido y una curva cerrada inferior que indujo al Árbol simple a confundir la masa inferior con el cuerpo de un 6. Random Forest, condicionado por los cortes aleatorios de píxeles, detectó trazos cruzados perpendiculares que lo llevaron a clasificarlo como 4. En contraste, k-NN calculó la vecindad global ponderada ($k = 5$), reconociendo correctamente la morfología del 2.

Para profundizar el análisis de fallos, en la Figura 8 se expone una galería con seis instancias de prueba complejas donde se suscitaron desacuerdos entre los modelos:

- **Muestra #8 (Real 9):** Árbol=7, RF=9, k-NN=7. El lazo superior angosto engañó al árbol y a k-NN confundiéndolo con un 7, mientras RF acertó.

- **Muestra #23 (Real 2):** Árbol=3, RF=2, k-NN=7. Trazo ondulado complejo resuelto correctamente por RF.
- **Muestra #99 (Real 3):** Árbol=7, RF=7, k-NN=7. Los tres modelos fallaron ante un dígito 3 con trazo superior excesivamente plano y alargado que asemeja un 7.
- **Muestra #119 (Real 4):** Árbol=9, RF=9, k-NN=9. Trazo cerrado en la cúspide que visualmente emula a un 9 para todos los clasificadores.
- **Muestra #154 (Real 0):** Árbol=1, RF=5, k-NN=5. Dígito 0 deforme y fragmentado con discontinuidades que desorientaron las fronteras de decisión.
- **Muestra #171 (Real 7):** Árbol=5, RF=7, k-NN=1. Dígito 7 con barra horizontal central que indujo a k-NN a predecir 1, mientras que RF preservó la clase verdadera.

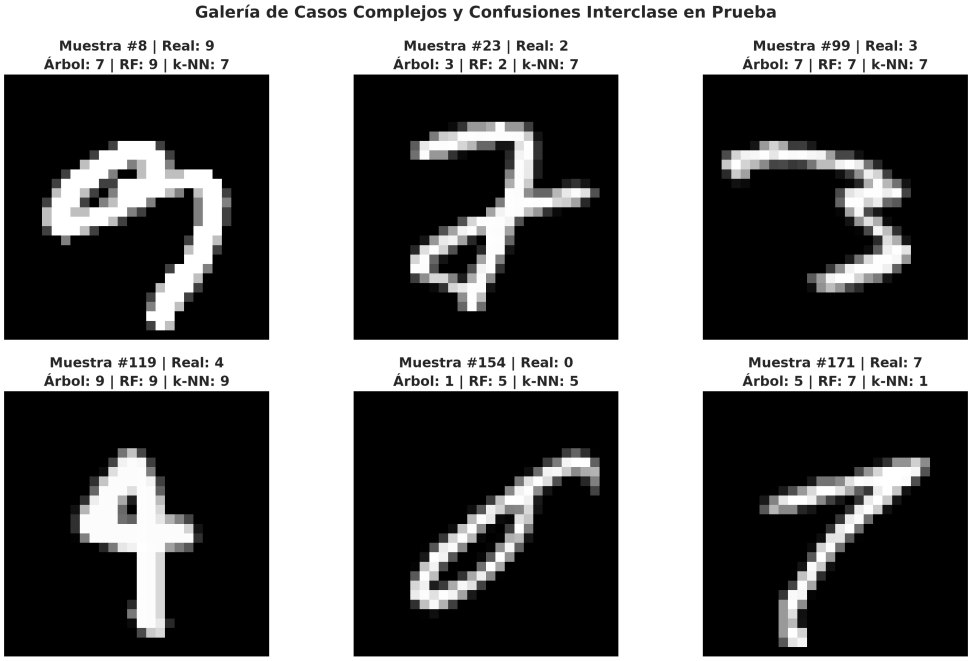


Figura 8: Galería de casos complejos y discrepancias interclase en el conjunto de prueba.

Respuestas a las Preguntas de Comparación:

1. **¿Qué modelo obtuvo la mayor exactitud y el menor error en prueba? Cuantifiquen la diferencia respecto al segundo, o indiquen si hubo empate.**

   El modelo con mejor desempeño global en el conjunto de prueba fue **Random Forest**, alcanzando una exactitud de **94.80 %** y un error mínimo de **5.20 %**. Superó a su competidor más cercano, el clasificador **k-NN** (exactitud de **94.55 %** y error de **5.45 %**), por una diferencia positiva de **0.25 puntos porcentuales** (p. p.). Ambos clasificadores superaron categóricamente al Árbol de decisión simple (81.80 % de exactitud y 18.20 % de error), situándose Random Forest **13.00 p. p.** por encima de este último.

2. **¿Qué modelo presentó la mayor brecha entre entrenamiento y prueba? Expliquen qué sugiere sin tomarla como prueba concluyente de sobreajuste. En k-NN, una observación de entrenamiento puede ser su propio vecino.**

   La mayor brecha de exactitud la presentó el **Árbol de decisión**, con una diferencia de **17.22 puntos porcentuales** (99.02 % en entrenamiento frente a 81.80 % en prueba). Esta notable brecha sugiere un claro sobreajuste (*overfitting*), en el cual el árbol individual memorizó patrones

de ruido y particularidades caligráficas locales del conjunto de entrenamiento al alcanzar una profundidad de 15 niveles sin podado.

En contraste, k-NN exhibió la menor brecha aparente (**1.96 p. p.**, con 96.51 % en entrenamiento y 94.55 % en prueba). No obstante, esta métrica debe interpretarse con cautela: cuando se evalúa el conjunto de entrenamiento en k-NN, cada instancia evaluada es su propio vecino más cercano a distancia euclidiana cero, lo que infla de forma intrínseca la exactitud de entrenamiento sin que signifique una generalización paramétrica subyacente.

3. **¿Qué confusión común o diferente observaron entre los modelos? Justifiquen qué clasificador recomendarían para esta tarea con al menos dos evidencias.**

   Se evidenciaron patrones de error característicos para cada modelo:

   - El Árbol de decisión concentra fallos severos en dígitos con contornos redondeados análogos (3 vs 5 con 14 casos, y 4 vs 9 con 11 casos).
   - Random Forest distribuye los errores de manera más homogénea y en magnitudes mucho menores; su peor caso fue 7 confundido con 9 (7 casos).
   - k-NN presentó confusiones asociadas a la dispersión de la distancia euclidiana en alta dimensión, confundiendo 2 con 7 (9 casos).

   Clasificador Recomendado: Se recomienda de manera concluyente el modelo **Random Forest**, fundamentado en dos evidencias sólidas:

   - *Evidencia 1 (Rendimiento predictivo superior):* Obtuvo la mayor exactitud en prueba (94.80 %) y el menor error (5.20 %), limitando la máxima confusión entre pares a tan solo 7 casos (frente a 14 del árbol y 9 de k-NN).
   - *Evidencia 2 (Eficiencia en inferencia):* Durante la predicción sobre las 2 000 muestras de prueba, Random Forest tardó apenas **0.62 segundos** (0.31 ms por muestra), mientras que k-NN requirió **2.65 segundos** (más de 4 veces más lento) debido al costo de calcular distancias a 10 000 vectores en 784 dimensiones. En un entorno productivo de digitalización masiva, Random Forest resulta mucho más escalable y rápido.

2.8 Habilidades blandas empleadas en la práctica

A continuación se indican las habilidades blandas aplicadas y las acciones concretas realizadas:

☐ **Liderazgo**

☒ **Trabajo en equipo:** Distribución coordinada de roles entre los integrantes (**Fiallos Yanza Jonathan Josue** y **Guevara Panimboza Steven Marlon**) para la estructuración y depuración modular del código en Python, supervisión de los entrenamientos y redacción rigurosa del informe técnico en LaTeX.

☒ **Comunicación asertiva:** Diálogo técnico continuo y fundamentado para consensuar la interpretación matemática de las matrices de confusión, la brecha de sobreajuste y la selección de argumentos de recomendación.

☐ **La empatía**

☒ **Pensamiento crítico:** Análisis reflexivo de las causas de error (maldición de la dimensionalidad en k-NN, memorización de datos en árboles simples y reducción de varianza en ensambles), yendo más allá de la simple transcripción de porcentajes.

☐ **Flexibilidad**

☐ **La resolución de conflictos**

☐ **Adaptabilidad**

☒ **Responsabilidad:** Cumplimiento estricto del protocolo experimental prescrito, respetando particiones estratificadas, fijación de semillas (`random_state=42`) y preservación del aislamiento del conjunto de prueba.

2.9 Conclusiones

1. **Desempeño en prueba:** Random Forest demostró ser el clasificador más robusto y preciso, alcanzando una exactitud de 94.80 % y una tasa de error de solo 5.20 % en el conjunto de prueba de 2 000 dígitos, superando a k-NN (94.55 %) y aventajando de manera contundente al Árbol de decisión simple (81.80 %) con una diferencia de 13.00 puntos porcentuales. Esto confirma experimentalmente la efectividad del aprendizaje por ensamble (*bagging*) para mitigar la varianza individual y capturar las variaciones en los trazos manuscritos.

2. **Diferencia entre entrenamiento y prueba:** El Árbol de decisión evidenció un severo sobreajuste (*overfitting*) con una brecha de generalización de 17.22 puntos porcentuales (99.02 % en entrenamiento vs 81.80 % en prueba), derivado de ramificaciones excesivamente profundas en un espacio de 784 variables. En contraste, Random Forest redujo sustancialmente esta brecha a 5.13 p. p. gracias a la aleatorización de características. Por su parte, la brecha aparentemente baja de k-NN (1.96 p. p.) refleja la condición particular del método en la que los puntos de entrenamiento coinciden consigo mismos a distancia cero, lo que enfatiza la necesidad de evaluar el rendimiento real exclusivamente con datos de prueba no vistos.

2.10 Recomendaciones

Para ensayos futuros, se propone implementar una optimización sistemática de hiperparámetros mediante **validación cruzada estratificada de k pliegues (*Stratified k-Fold Cross Validation*)** con $k = 5$, aplicada **exclusivamente sobre el conjunto de entrenamiento (X_{ent})** mediante `GridSearchCV` o técnicas bayesianas.

De este modo, se podrán explorar y sintonizar parámetros clave (como `n_estimators`, `max_depth` y criterios de poda `ccp_alpha` en Random Forest, así como diferentes métricas de distancia en k-NN) sin incurrir en fuga de información (*data leakage*), manteniendo el conjunto de prueba de 2 000 observaciones completamente aislado hasta la evaluación final del modelo definitivo. Asimismo, se aconseja evaluar la aplicación de Análisis de Componentes Principales (PCA) como paso previo para reducir la dimensionalidad de 784 a unas 50 componentes, lo cual aceleraría drásticamente la velocidad de inferencia de k-NN sin comprometer sustancialmente la exactitud.

2.11 Referencias bibliográficas

Referencias

[1] Y. LeCun, C. Cortes y C. J. C. Burges, «mnist_784», v. 1, OpenML, ID 554. [En línea]. Disponible: https://www.openml.org/d/554

[2] Scikit-learn developers, «DecisionTreeClassifier», *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

[3] Scikit-learn developers, «RandomForestClassifier», *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[4] Scikit-learn developers, «KNeighborsClassifier», *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

[5] Scikit-learn developers, «accuracy_score», *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

2.12 Anexos

Anexo A. Código de partida para preparar los datos A continuación se transcribe el bloque de partida provisto en la guía para la ingesta y partición del dataset:

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

X, y = fetch_openml(data_id=554, as_frame=False, return_X_y=True)
X = X.astype("float32") / 255.0
y = y.astype("int64")

def tomar_muestra(X_base, y_base, n):
    X_muestra, _, y_muestra, _ = train_test_split(
        X_base, y_base, train_size=n, stratify=y_base,
        random_state=42
    )
    return X_muestra, y_muestra

X_ent, y_ent = tomar_muestra(X[:60000], y[:60000], 10000)
X_pru, y_pru = tomar_muestra(X[60000:], y[60000:], 2000)
print(X_ent.shape, X_pru.shape) # (10000, 784), (2000, 784)
```

Anexo B. Archivos para la entrega De acuerdo a lo estipulado en la guía, se generaron y entregan los siguientes artefactos:

- **Informe Técnico:** Documento formal en formato LaTeX (`informe_mnist.tex`) y su versión compilada en PDF (`informe_mnist.pdf`), conteniendo la portada institucional de la UTA, la Tabla 2 con todas las métricas requeridas, las matrices de confusión analizadas, el ejemplo de error, la comparativa técnica fundamentada, habilidades blandas, conclusiones y recomendaciones.
- **Cuaderno interactivo de Jupyter (`.ipynb`):** Archivo `Taller_APE_MNIST_Arbol_RandomForest_kNN.ipynb` totalmente ejecutado, con todas las salidas, celdas de código reproducibles y visualizaciones embebidas desde la carga de MNIST hasta las matrices de confusión finales.